1 Assignment 01 - EN3160

Fernando H.M.D. - 230180F

Github Repo Link - https://github.com/Minu-Fernando/EN3160.git

1.1 Question 01

[1]:
```
# Intensity transformation functions
import cv2
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
IMAGE_DIR = Path(r"C:\Users\minay\Desktop\ENTC\ACA\Semester 5\EN3160 - Image Processing and Machine␣
 ↪Vision\Assignment\EN3160\Assignment 1\images")

def load_image(filename, flags):
    image = cv2.imread(str(IMAGE_DIR / filename), flags)
    if image is None:
        raise FileNotFoundError("Image could not be loaded. Check the file path.")
    return image

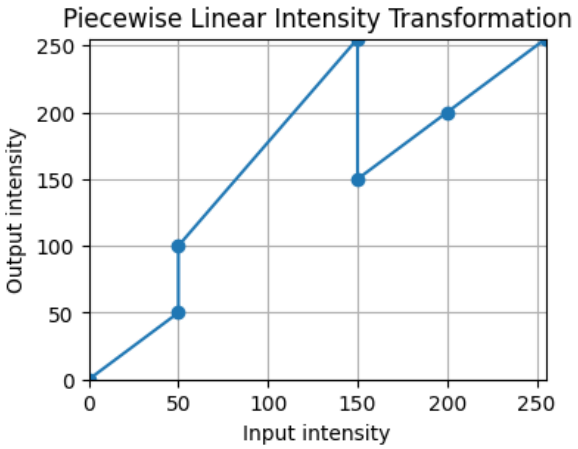
img = load_image("q1.jpeg", cv2.IMREAD_GRAYSCALE)

def intensity_transform(im, breakpoints):
    breakpoints=np.asarray(breakpoints,dtype=np.float32)
    x=breakpoints[:,0]; y=breakpoints[:,1]
    im_float=im.astype(np.float32); transformed=np.zeros_like(im_float)
    lut=np.zeros(256,dtype=np.float32); intensity_values=np.arange(256,dtype=np.float32)
    for i in range(len(breakpoints)-1):
        x1,y1=breakpoints[i]; x2,y2=breakpoints[i+1]
        if x1==x2: continue
        mask=(intensity_values>=x1)&(intensity_values<=x2)
        lut[mask]=y1+(y2-y1)*(intensity_values[mask]-x1)/(x2-x1)
    for i in range(len(breakpoints)-1):
        if x[i]==x[i+1]:
            jump_x=int(x[i]); lut[jump_x]=y[i+1]
    lut=np.clip(lut,0,255).astype(np.uint8)
    return lut[im]

def show_piecewise_result(original, breakpoints, transformed):
    plt.figure(figsize=(4, 3))
    plt.plot(breakpoints[:, 0], breakpoints[:, 1], marker='o')
    plt.xlim(0, 255); plt.ylim(0, 255)
    plt.xlabel("Input intensity"); plt.ylabel("Output intensity")
    plt.title("Piecewise Linear Intensity Transformation"); plt.grid(True); plt.show()
    fig, axes = plt.subplots(1, 2, figsize=(6, 3))
    axes[0].imshow(original, cmap="gray", vmin=0, vmax=255); axes[0].set_title("Original Image")
    axes[1].imshow(transformed, cmap="gray", vmin=0, vmax=255); axes[1].set_title("Transformed Image")
    for ax in axes: ax.axis("off")
    plt.tight_layout(); plt.show()

breakpoints=np.array([[0,0],[50,50],[50,100],[150,255],[150,150],[200,200],[255,255]])
transformed=intensity_transform(img,breakpoints)

show_piecewise_result(img, breakpoints, transformed)
```

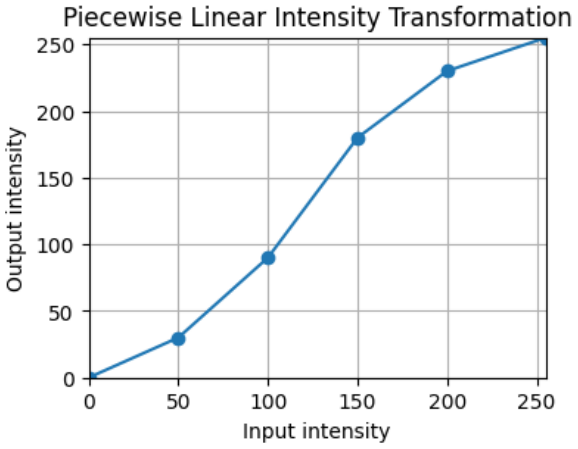




For a visually pleasing output

[2]:
```
breakpoints=np.array([[0,0],[50,30],[100,90],[150,180],[200,230],[255,255]])
transformed=intensity_transform(img,breakpoints)

show_piecewise_result(img, breakpoints, transformed)
```





1.2 Question 2

[3]:
```
img = load_image("q2.jpeg", cv2.IMREAD_GRAYSCALE)
```

White matter and Gray matter in a brain proton density image does not mean the places with high pixel intensities and lower areas. rather, White and Gray matter means seperate areas of tissue.
To find the pixel intensities of the designated areas, we can inspect biological areas of white and gray matter. We can inspect that the gray matter is in the region of Y[80:90], X[30:40]... the wrinkly outer tissue of the brain. And the white matter is roughly Y[100:120] X[50:70] region. The white range in the middle is not white matter. that is CSF region.

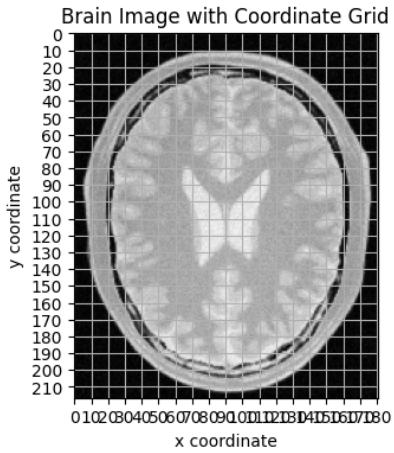
[4]:
```
import matplotlib.pyplot as plt
plt.figure(figsize=(4,4)); plt.imshow(img,cmap="gray",vmin=0,vmax=255)
plt.xticks(range(0,img.shape[1],10)); plt.yticks(range(0,img.shape[0],10)); plt.grid(True)
plt.xlabel("x coordinate"); plt.ylabel("y coordinate"); plt.title("Brain Image with Coordinate Grid"); plt.show()

white_roi = img[100:120, 50:60]

print("White matter mean:", np.mean(white_roi))
print("White matter min :", np.min(white_roi))
print("White matter max :", np.max(white_roi))

gray_roi = img[150:160, 40:50]

print("Gray matter mean:", np.mean(gray_roi))
print("Gray matter min :", np.min(gray_roi))
print("Gray matter max :", np.max(gray_roi))
```




```
White matter mean: 167.21
White matter min : 148
```

```
White matter max : 194
Gray matter mean: 196.85
Gray matter min : 171
Gray matter max : 222
```

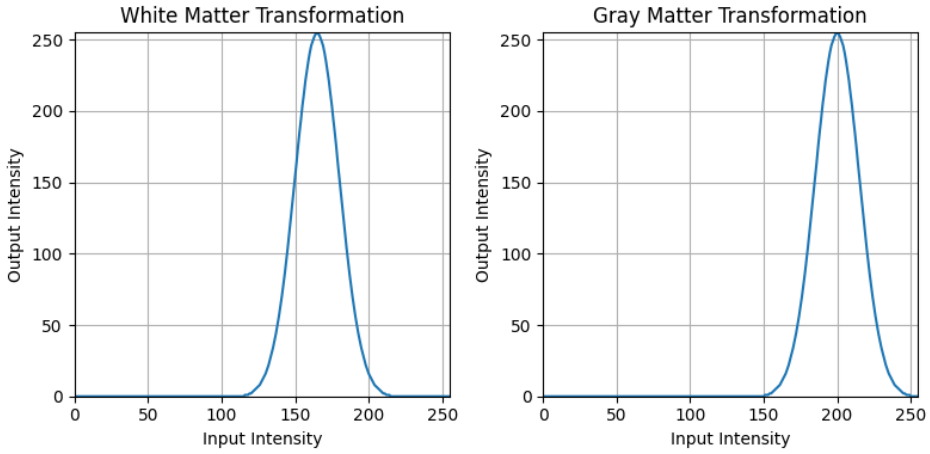
Now, I am using Gaussian pulse as the intensity transformation function to accentuate only the required range of pixel intensities. The reason to use Gaussian pulse rather than block pixels is to remove the jump discontinuities and obtain a smooth contrast from the rest of the image. The white matter is centered arounf 167.21 ~ 165 and gray matter is centered arounf 196.85 ~ 200. Such is the value of miu. As for sigma, sigma reflects the range of values that will be accentuated.

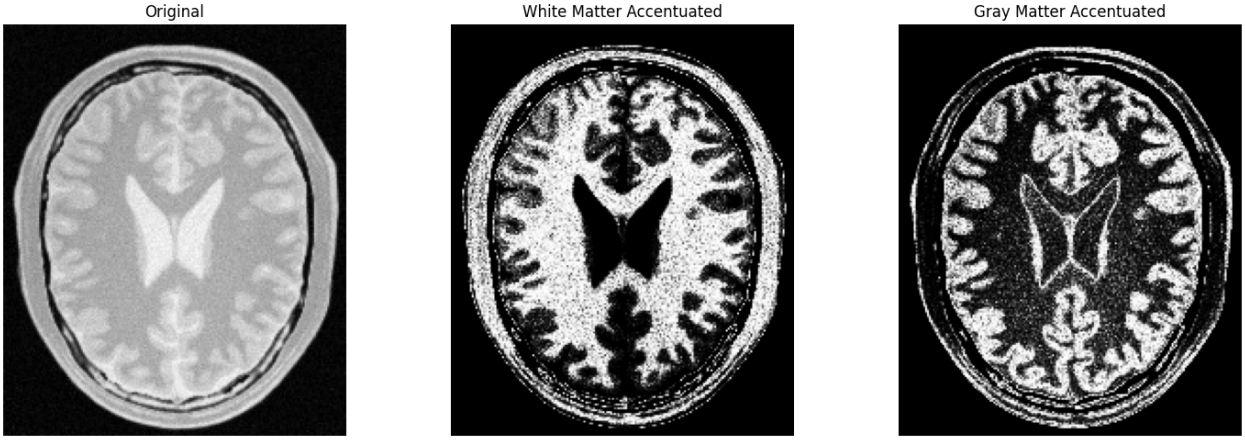
[5]:
```
def gaussian_intensity_transform(im,mu,sigma):
    x=np.arange(256)
    T=255*np.exp(-((x-mu)**2)/(2*sigma**2))
    T=np.clip(T,0,255).astype(np.uint8)
    transformed=T[im]
    return transformed,T

white_matter,T_white=gaussian_intensity_transform(img,mu=165,sigma=15)
gray_matter,T_gray=gaussian_intensity_transform(img,mu=200,sigma=15)

x=np.arange(256)
fig,axes=plt.subplots(1,2,figsize=(8,4))
axes[0].plot(x,T_white); axes[0].set_title("White Matter Transformation"); axes[0].set_xlabel("Input Intensity");␣
 ↪axes[0].set_ylabel("Output Intensity"); axes[0].set_xlim(0,255); axes[0].set_ylim(0,255); axes[0].grid()
axes[1].plot(x,T_gray); axes[1].set_title("Gray Matter Transformation"); axes[1].set_xlabel("Input Intensity"); axes[1].
 ↪set_ylabel("Output Intensity"); axes[1].set_xlim(0,255); axes[1].set_ylim(0,255); axes[1].grid()
plt.tight_layout(); plt.show()

fig,axes=plt.subplots(1,3,figsize=(15,5))
axes[0].imshow(img,cmap="gray"); axes[0].set_title("Original")
axes[1].imshow(white_matter,cmap="gray"); axes[1].set_title("White Matter Accentuated")
axes[2].imshow(gray_matter,cmap="gray"); axes[2].set_title("Gray Matter Accentuated")
for ax in axes: ax.axis("off")
plt.tight_layout(); plt.show()
```





1.3 Question 03

In this question, we talk about L* a* b* colour images. In an RGB image, the brightness of the image is not an attribute that i controlled by just R, g or B, but by the combination of all three. So, doing a gamma correction on an RGB image can alter the colours of the image. A way to prevent this is to, apply gamma correction in the Lab plane. In Lab plane, the image is segragrated to

$$L* = lightness$$

$$a^* = \text{green} \leftrightarrow \text{red}$$

$$b* = blue \leftrightarrow yellow$$

Then we apply gamma to the L* plane.

[6]:
```
img3 = load_image("q3.jpeg", cv2.IMREAD_COLOR_BGR)

img3_lab = cv2.cvtColor(img3, cv2.COLOR_BGR2LAB)
img3_rgb = cv2.cvtColor(img3, cv2.COLOR_BGR2RGB)
L, a, b = cv2.split(img3_lab)

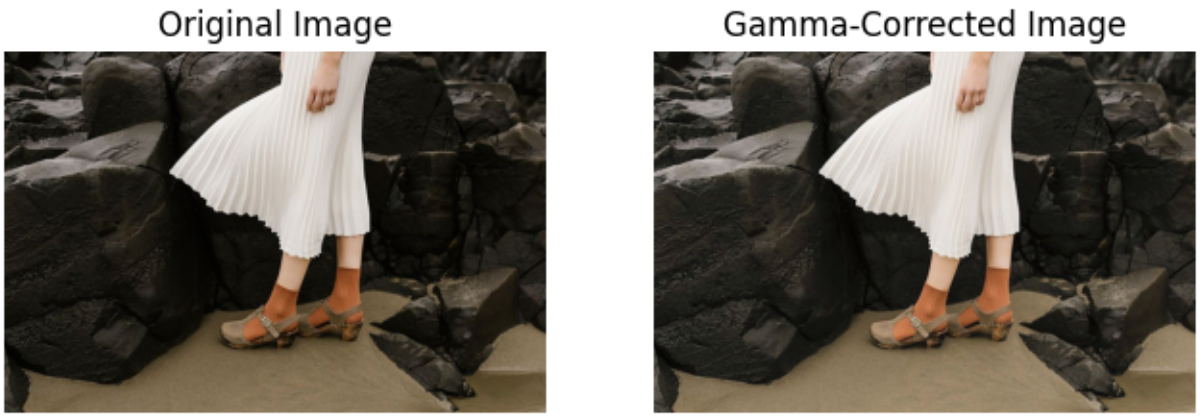
# Applying gamma correction to the L place.
gamma = 0.8
lut = np.array([((i / 255.0) ** gamma) * 255 for i in range(256)], dtype=np.uint8)

L_modified = cv2.LUT(L, lut)

# Merge back and convert to BGR
corrected_lab = cv2.merge([L_modified, a, b])
corrected_bgr = cv2.cvtColor(corrected_lab, cv2.COLOR_LAB2BGR)
corrected_rgb = cv2.cvtColor(corrected_lab, cv2.COLOR_LAB2RGB)

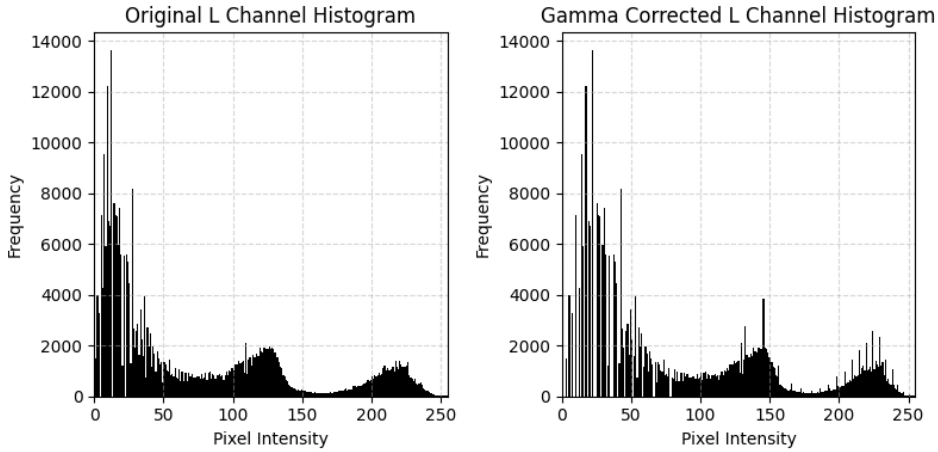
fig,axes=plt.subplots(1,2,figsize=(8,6)); colors=('r','g','b')
axes[0].imshow(img3_rgb); axes[0].set_title("Original Image"); axes[0].axis("off")
axes[1].imshow(corrected_rgb); axes[1].set_title("Gamma-Corrected Image"); axes[1].axis("off")
```

[6]:
```
(np.float64(-0.5), np.float64(719.5), np.float64(479.5), np.float64(-0.5))
```



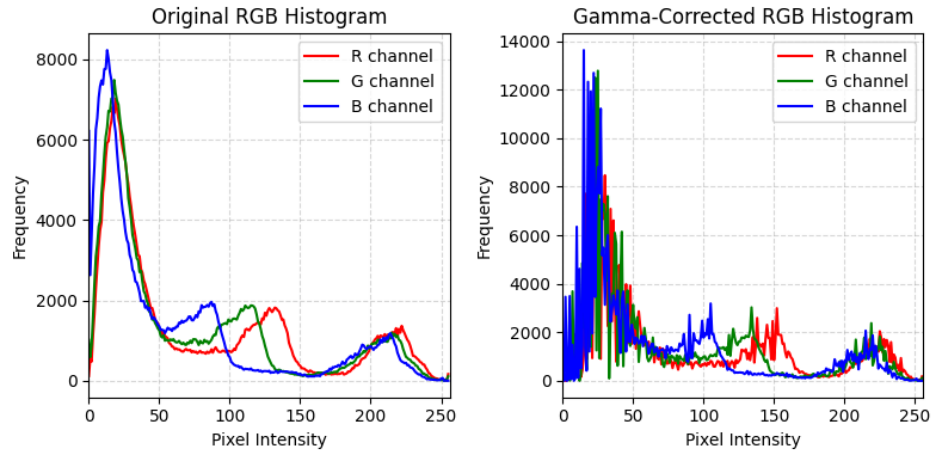
The gamma correction has made the darker areas specially the rock, more detailed and less shadowy.

[7]:
```
fig,axes=plt.subplots(1,2,figsize=(8,4))
axes[0].hist(L.ravel(),bins=256,range=[0,256],color='black'); axes[0].set_title("Original L Channel Histogram");␣
 ↪axes[0].set_xlabel("Pixel Intensity"); axes[0].set_ylabel("Frequency"); axes[0].set_xlim(0,255); axes[0].
 ↪grid(True,linestyle='--',alpha=0.5)
axes[1].hist(L_modified.ravel(),bins=256,range=[0,256],color='black'); axes[1].set_title("Gamma Corrected L Channel␣
 ↪Histogram"); axes[1].set_xlabel("Pixel Intensity"); axes[1].set_ylabel("Frequency"); axes[1].set_xlim(0,255); axes[1].
 ↪grid(True,linestyle='--',alpha=0.5)
plt.tight_layout(); plt.show()
```



We can clearly see the effect of the gamma correction in the second histogram, where the low intensity pixels are mapped into a ligheter range. That is, the darker areas are lightened.

[8]:
```
fig,axes=plt.subplots(1,2,figsize=(8,4))
for i,col in enumerate(colors):
    hist=cv2.calcHist([img3_rgb],[i],None,[256],[0,256]); axes[0].plot(hist,color=col,label=f"{col.upper()} channel")
axes[0].set_title("Original RGB Histogram"); axes[0].set_xlabel("Pixel Intensity"); axes[0].set_ylabel("Frequency");␣
 ↪axes[0].set_xlim([0,256]); axes[0].legend(); axes[0].grid(True,linestyle="--",alpha=0.5)
for i,col in enumerate(colors):
    hist=cv2.calcHist([corrected_rgb],[i],None,[256],[0,256]); axes[1].plot(hist,color=col,label=f"{col.upper()}␣
 ↪channel")
axes[1].set_title("Gamma-Corrected RGB Histogram"); axes[1].set_xlabel("Pixel Intensity"); axes[1].
 ↪set_ylabel("Frequency"); axes[1].set_xlim([0,256]); axes[1].legend(); axes[1].grid(True,linestyle="--",alpha=0.5)
plt.tight_layout(); plt.show()
```



Interpretation of results: Gamma correction is a nonlinear transformation. It is going to transform the L and the RGB planes non linearly. This accounts for the noisy like histograms.

1.4 Question 04

In this question, we split the image into, Hue, Saturation and Value/ Brightness planes. Instead of directly chandinng the value of brightness, we apply a gaussian smoothing function centered around 128, with a variance of 70 to the Saturation plane.

Saturation has higher values for high colour areas ex: the spider man suit, zendaya hair where as lower values for less colourfull areas ex: buildings. Thus, applying the transformatio to the saturation plane,enhances the visibility of colours that is, makes the image look more colourful/vivid.

[9]:
```
img4 = load_image("q4.jpeg", cv2.IMREAD_COLOR_RGB)

# Splitting into Saturation, Hue and Value planes
img4_HSV = cv2.cvtColor(img4, cv2.COLOR_RGB2HSV)
H, S, V = cv2.split(img4_HSV)
```

The value of (a) determines the strength of the vibrance enhancement applied to the saturation plane. When (a=0), the transformation reduces to the identity transformation. As (a) increases, the saturation of the image increases, making the existing colours appear more vivid while preserving their hue.

[10]:
```
def intensity_transform(x, a, sigma):
    x = np.array(x, dtype=np.float64)

    gaussian_term = a * 128.0 * np.exp(-((x - 128.0) ** 2) / (2.0 * (sigma ** 2)))
    transformed = x + gaussian_term

    result = np.minimum(transformed, 255.0)

    return np.round(result).astype(np.uint8)

# Apply the transformation to the Saturation channel
saturated_image = intensity_transform(S, a=0.65, sigma=70)

corrected_hsv = cv2.merge([H, saturated_image, V])
corrected_rgb = cv2.cvtColor(corrected_hsv, cv2.COLOR_HSV2RGB)
```

After careful tryouts

$$a = 0.65$$

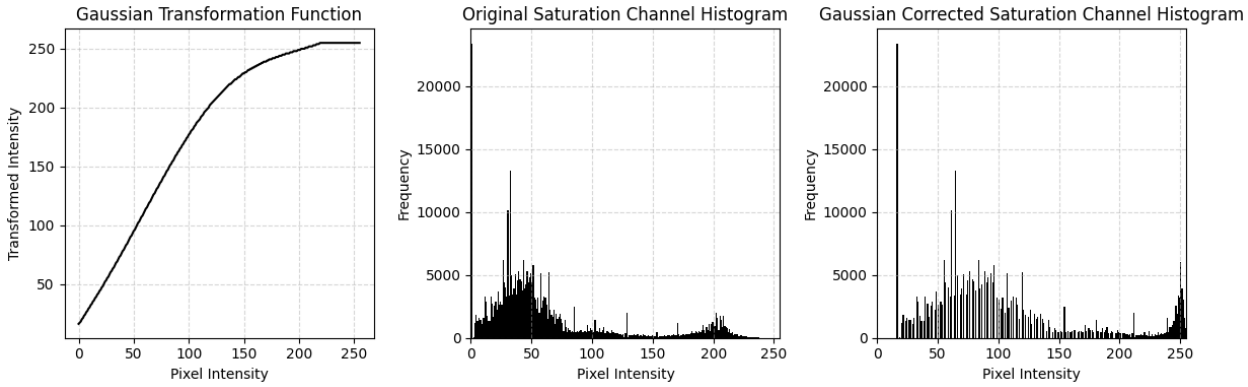
was selected as the best vlue for the image to look visually pleasing without having extreme colour contrasts.
In the image, it can be seen that the red colour of the suit and orange of hair are more highlighted than the original image. This is the difference done by the transformation.

[11]:
```
fig,axes=plt.subplots(1,2,figsize=(8,4))
axes[0].imshow(img4); axes[0].set_title("Original Image"); axes[0].axis("off")
axes[1].imshow(corrected_rgb); axes[1].set_title("Gaussian-Corrected Image"); axes[1].axis("off")

fig,axes=plt.subplots(1,3,figsize=(12,4))
axes[0].plot(np.arange(256),intensity_transform(np.arange(256),a=0.65,sigma=70),color='black'); axes[0].
 ↪set_title("Gaussian Transformation Function"); axes[0].set_xlabel("Pixel Intensity"); axes[0].
 ↪set_ylabel("Transformed Intensity"); axes[0].grid(True,linestyle='--',alpha=0.5)
axes[1].hist(S.ravel(),bins=256,range=[0,256],color='black'); axes[1].set_title("Original Saturation Channel␣
 ↪Histogram"); axes[1].set_xlabel("Pixel Intensity"); axes[1].set_ylabel("Frequency"); axes[1].set_xlim(0,255); axes[1].
 ↪grid(True,linestyle='--',alpha=0.5)
axes[2].hist(saturated_image.ravel(),bins=256,range=[0,256],color='black'); axes[2].set_title("Gaussian Corrected␣
 ↪Saturation Channel Histogram"); axes[2].set_xlabel("Pixel Intensity"); axes[2].set_ylabel("Frequency"); axes[2].
 ↪set_xlim(0,255); axes[2].grid(True,linestyle='--',alpha=0.5)
plt.tight_layout(); plt.show()
```



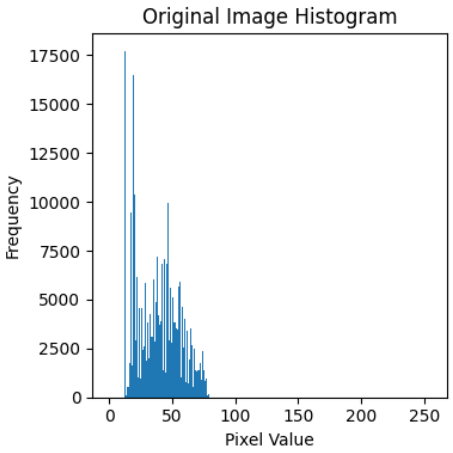




1.5 Question 05

[12]:
```
img5 = load_image("q5.tif", cv2.IMREAD_GRAYSCALE)

hist,bins=np.histogram(img5.flatten(),256,[0,256])
fig,axes=plt.subplots(1,1,figsize=(4,4)); plt.bar(x,hist,width=1); plt.xlabel('Pixel Value'); plt.ylabel('Frequency');␣
↪plt.title('Original Image Histogram'); plt.tight_layout(); plt.show()
```



We can see that the image has more pixels in the range of low pixel values. Now let us apply histogram equalization.

$$s_k = T(r_k) = \frac{L-1}{MN}\sum_{j=0}^{k} n_j \tag{1}$$

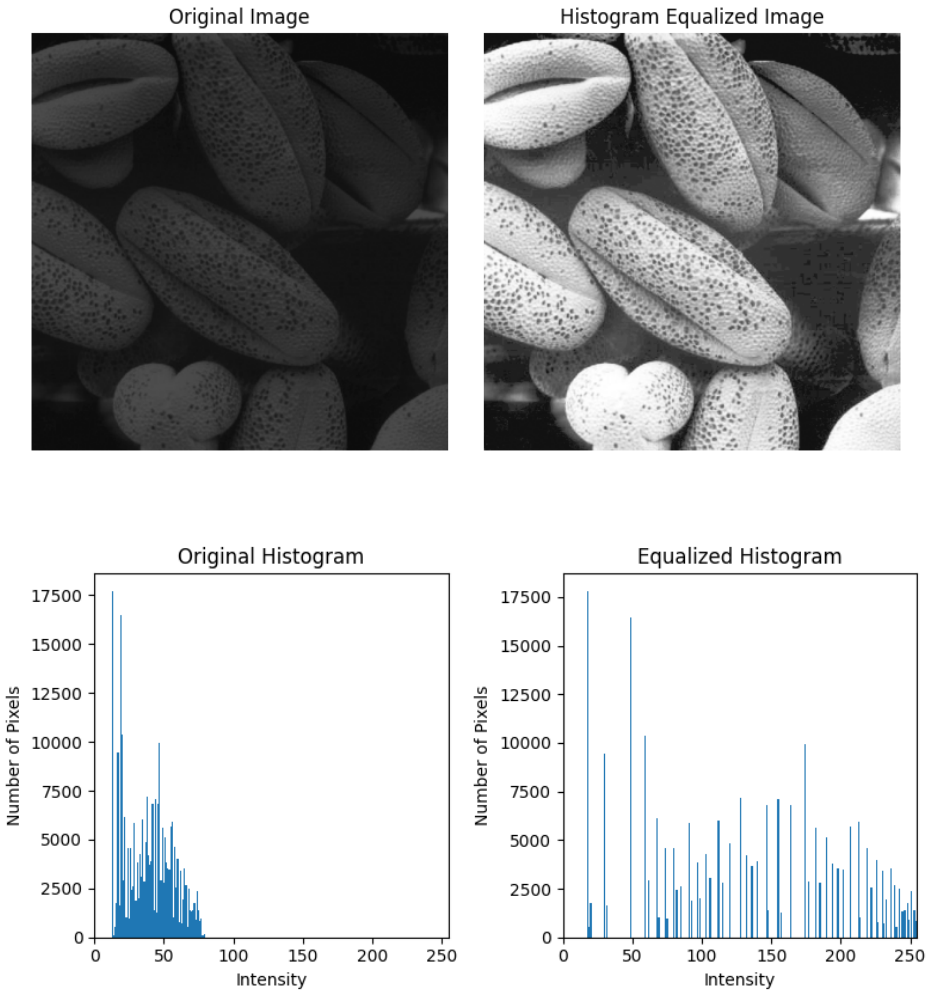
[13]:
```
def histogram_equalization(img):
    hist=np.zeros(256,dtype=np.int64)
    for pixel in img.ravel(): hist[pixel]+=1
    cdf=np.zeros(256,dtype=np.float64)
    for i in range(256): cdf[i]=np.sum(hist[:i+1]); cdf[i]=cdf[i]/img.size
    mapping=np.round(255*cdf).astype(np.uint8)
    equalized=mapping[img]
    return equalized

img5_eq=histogram_equalization(img5)
hist_equalized=np.zeros(256,dtype=np.int64)
for pixel in img5_eq.ravel(): hist_equalized[pixel]+=1

fig,axes=plt.subplots(1,2,figsize=(8,4))
axes[0].imshow(img5,cmap="gray",vmin=0,vmax=255); axes[0].set_title("Original Image"); axes[0].axis("off")
axes[1].imshow(img5_eq,cmap="gray",vmin=0,vmax=255); axes[1].set_title("Histogram Equalized Image"); axes[1].axis("off")
plt.tight_layout(); plt.show()

x=np.arange(256)
```

```
fig,axes=plt.subplots(1,2,figsize=(8,4))
axes[0].bar(x,hist,width=1); axes[0].set_title("Original Histogram"); axes[0].set_xlabel("Intensity"); axes[0].
 ↪set_ylabel("Number of Pixels"); axes[0].set_xlim(0,255)
axes[1].bar(x,hist_equalized,width=1); axes[1].set_title("Equalized Histogram"); axes[1].set_xlabel("Intensity");␣
 ↪axes[1].set_ylabel("Number of Pixels"); axes[1].set_xlim(0,255)
plt.tight_layout(); plt.show()
```



Histogram equalization spreads out the pixel values more evenly, especially when they were previously packed into a smaller range of intensities.

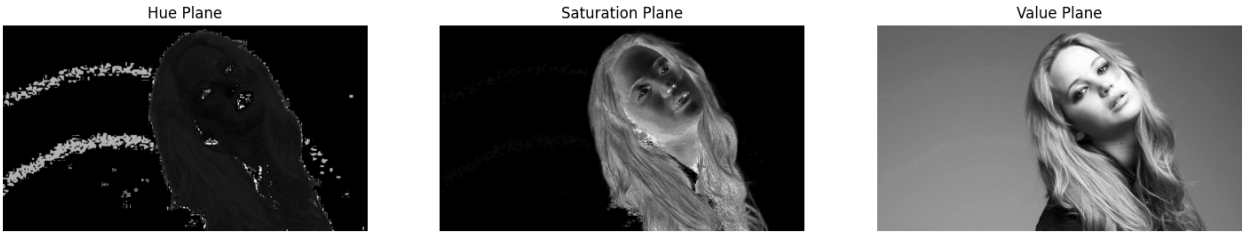
However, this does not mean that every histogram bin will be filled or that every bin will have the same frequency. It only gives a rough equalization of the intensity distribution.

1.6 Question 06

Histogram Equalization of foreground

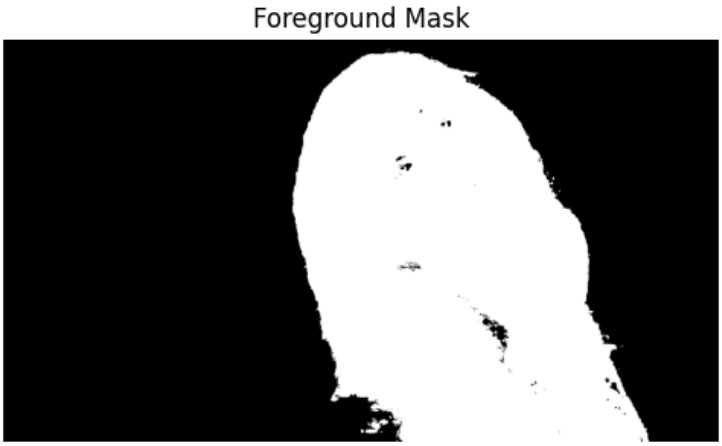
[14]:
```
img6 = load_image("q6.jpeg", cv2.IMREAD_COLOR_RGB)

img6_HSV=cv2.cvtColor(img6,cv2.COLOR_RGB2HSV); H,S,V=cv2.split(img6_HSV)
fig,axes=plt.subplots(1,3,figsize=(18,4))
axes[0].imshow(H,cmap='gray'); axes[0].set_title('Hue Plane')
axes[1].imshow(S,cmap='gray'); axes[1].set_title('Saturation Plane')
axes[2].imshow(V,cmap='gray'); axes[2].set_title('Value Plane')
for ax in axes: ax.axis('off')
plt.show()
```



From this image, we can deduce that the saturation plane focuses more on the foreground image. So, for the question, to apply histogram equalization to the foreground, we pick the saturation plane.
Now, lets mask it and threshold.

[15]:
```
_,mask=cv2.threshold(S,13,255,cv2.THRESH_BINARY); hist,bins=np.histogram(mask.flatten(),256,[0,256])
plt.figure(figsize=(5, 5))
plt.imshow(mask,cmap="gray"); plt.title("Foreground Mask"); plt.axis("off"); plt.tight_layout(); plt.show()
```



After experimenting on the value of the threold of the mask, i deduced on 13 as the lower threshold and 255 as the upper threshold to capture all of the foreground of the image.

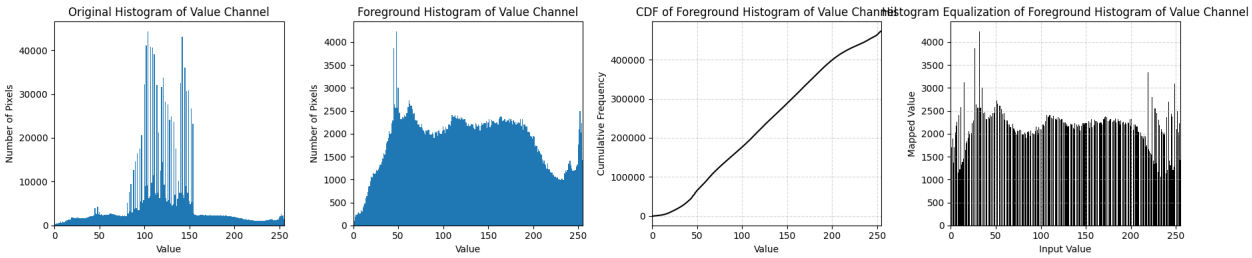
[16]:
```
foreground=cv2.bitwise_and(img6,img6,mask=mask); hist,bins=np.histogram(foreground.flatten(),256,[0,256])
plt.figure(figsize=(5, 5))
plt.imshow(foreground); plt.title("Foreground Mask"); plt.axis("off"); plt.tight_layout(); plt.show()
```



This shows the foreground extracted image - when using the cv.bitwise_and, the background values all becomes 0. Only the foreground values stay preserved.
Now, we can compare the histograms of value/brightness before and after the cv.bitwise_and.

We compare the value and equalize that, so that the colours do not get distort.

[17]:
```
hist,bins=np.histogram(V.flatten(),256,[0,256])
H_f,S_f,V_f=cv2.split(cv2.cvtColor(foreground,cv2.COLOR_RGB2HSV)); foreground_values=V_f[mask==255]
hist_f,bins_f=np.histogram(foreground_values,bins=256,range=(0,256)); cdf_f=np.cumsum(hist_f)
total_fg_pixels=cdf_f[-1]; lut=np.round((255.0/total_fg_pixels)*cdf_f).astype(np.uint8)
V_eq=np.zeros_like(V_f,dtype=np.uint8); V_eq[mask>0]=lut[V_f[mask>0]]; equalized_values=V_eq[mask>0]; hist_eq,_=np.
 ↪histogram(equalized_values,bins=256,range=(0,256))
fig,axes=plt.subplots(1,4,figsize=(18,4)); x=np.arange(256)
axes[0].bar(x,hist,width=1); axes[0].set_title("Original Histogram of Value Channel"); axes[0].set_xlabel("Value");␣
 ↪axes[0].set_ylabel("Number of Pixels"); axes[0].set_xlim(0,255)
axes[1].bar(x,hist_f,width=1); axes[1].set_title("Foreground Histogram of Value Channel"); axes[1].set_xlabel("Value");␣
 ↪axes[1].set_ylabel("Number of Pixels"); axes[1].set_xlim(0,255)
axes[2].plot(cdf_f,color='black'); axes[2].set_title("CDF of Foreground Histogram of Value Channel"); axes[2].
 ↪set_xlabel("Value"); axes[2].set_ylabel("Cumulative Frequency"); axes[2].set_xlim(0,255); axes[2].
 ↪grid(True,linestyle='--',alpha=0.5)
axes[3].bar(range(256),hist_eq,color='black'); axes[3].set_title("Histogram Equalization of Foreground Histogram of␣
 ↪Value Channel"); axes[3].set_xlabel("Input Value"); axes[3].set_ylabel("Mapped Value"); axes[3].set_xlim(0,255);␣
 ↪axes[3].grid(True,linestyle='--',alpha=0.5)
plt.tight_layout(); plt.show()
```

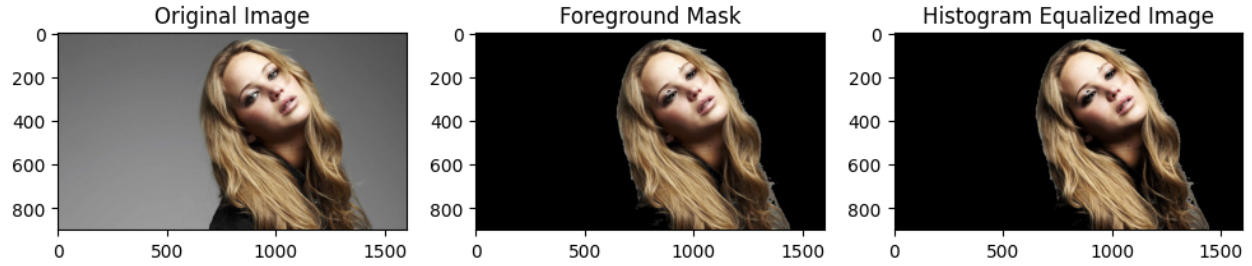


To get the foreground histogram, only the filtered non zero values are considered. Otherwise, there will be a too big of a spike at zero and the rest would not be intelligible.

Now, lets apply histogram equalization.

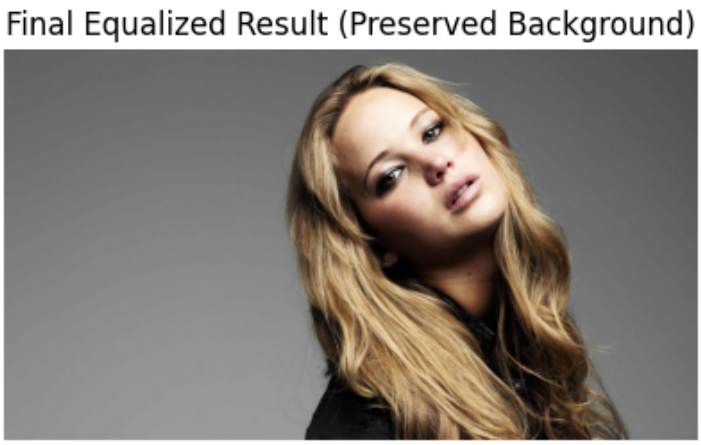
Now we can see the histogram equalized V channe of the image. Only the foreground pixels are equalized. The background pixels are all 0. They are filtered out.

[18]:
```
Equalized_HSV=cv2.merge([H,S,V_eq]); Equalized_RGB=cv2.cvtColor(Equalized_HSV,cv2.COLOR_HSV2RGB)
fig,axes=plt.subplots(1,3,figsize=(12,4))
axes[0].imshow(img6); axes[0].set_title("Original Image")
axes[1].imshow(foreground); axes[1].set_title("Foreground Mask")
axes[2].imshow(Equalized_RGB); axes[2].set_title("Histogram Equalized Image")
plt.show()
```



The image shows the original image of jennifer and her foreground image and the histogram equalized foreground image.

[19]:
```
background=cv2.bitwise_and(img6,img6,mask=cv2.bitwise_not(mask)); foreground_equalized=Equalized_RGB
final_image=cv2.add(foreground_equalized,background)
plt.figure(figsize=(5,5)); plt.imshow(final_image); plt.title("Final Equalized Result (Preserved Background)"); plt.
 ↪axis("off"); plt.show()
```



This is the final equalised foreground with background.

1.7 Question 07

In this question we are going to test sobel filtering.
Sobel filter is mainly used to identify edges. The sobel filter kernal is

$$[[1, 0, -1], [2, 0, -2], [1, 0, -1]]$$

this is the verticle sobel filter, and can be used to identify horizontal edges.

[20]:
```
img7 = load_image("q7.jpeg", cv2.IMREAD_GRAYSCALE)

kernalx=np.array([[-1,0,1],[-2,0,2],[-1,0,1]]); kernaly=np.array([[1,2,1],[0,0,0],[-1,-2,-1]])
sobel_x=cv2.filter2D(img7,ddepth=cv2.CV_64F,kernel=kernalx); sobel_y=cv2.filter2D(img7,ddepth=cv2.CV_64F,kernel=kernaly)
fig,axes=plt.subplots(1,2,figsize=(8,4)); axes[0].imshow(sobel_x,cmap='gray'); axes[0].set_title('Sobel Edge Detection␣
 ↪- X Direction'); axes[1].imshow(sobel_y,cmap='gray'); axes[1].set_title('Sobel Edge Detection - Y Direction')
plt.tight_layout(); plt.show()

# taking abs instead of clipping at zero can detect more edges, becuase clipping at zero disregards all negative␣
 ↪gradients and abs at least potraits them without hiding them.
# or, we can directly use imshow, which would map the minimum and maximum values to 0 to 255 making it possible to show␣
 ↪more edges,
# but it would not be a good representation of the actual gradient values.

def manual_filter2D(img,kernel):
    h,w=img.shape; kh,kw=kernel.shape; pad_h=kh//2; pad_w=kw//2
    padded=np.pad(img,((pad_h,pad_h),(pad_w,pad_w)),mode='constant'); output=np.zeros((h,w),dtype=np.float64)
    for y in range(h):
        for x in range(w):
            region=padded[y:y+kh,x:x+kw]; output[y,x]=np.sum(region*kernel)
    return output

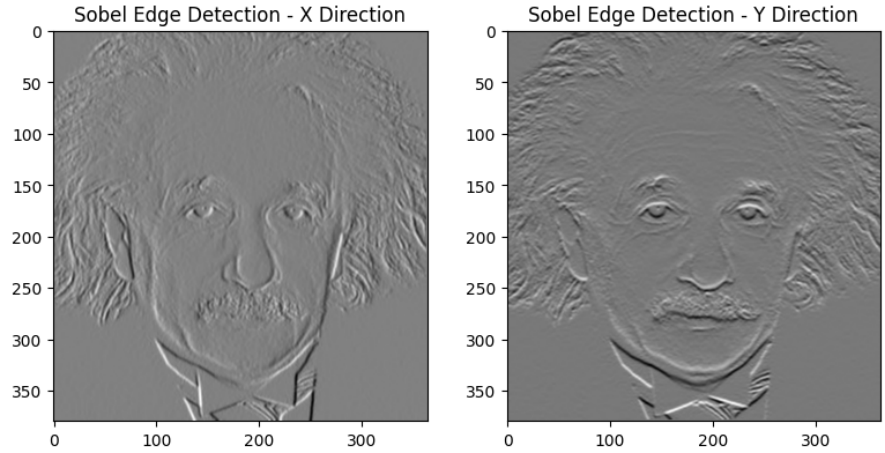
sobel_x_manual=manual_filter2D(img7,kernalx); sobel_y_manual=manual_filter2D(img7,kernaly)

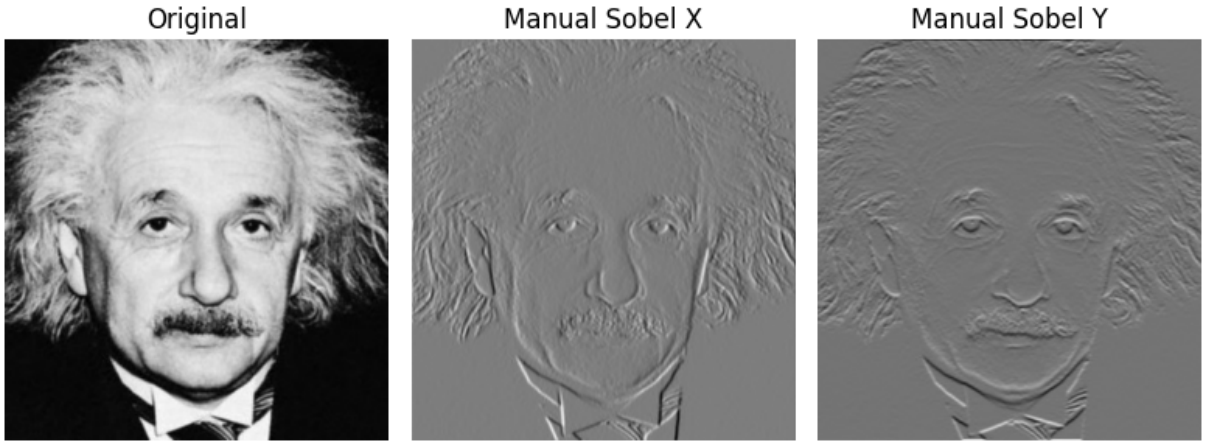
fig,axes=plt.subplots(1,3,figsize=(8,4)); axes[0].imshow(img7,cmap='gray'); axes[0].set_title('Original'); axes[1].
 ↪imshow(sobel_x_manual,cmap='gray'); axes[1].set_title('Manual Sobel X'); axes[2].imshow(sobel_y_manual,cmap='gray');␣
 ↪axes[2].set_title('Manual Sobel Y')
for ax in axes: ax.axis('off')
plt.tight_layout(); plt.show()

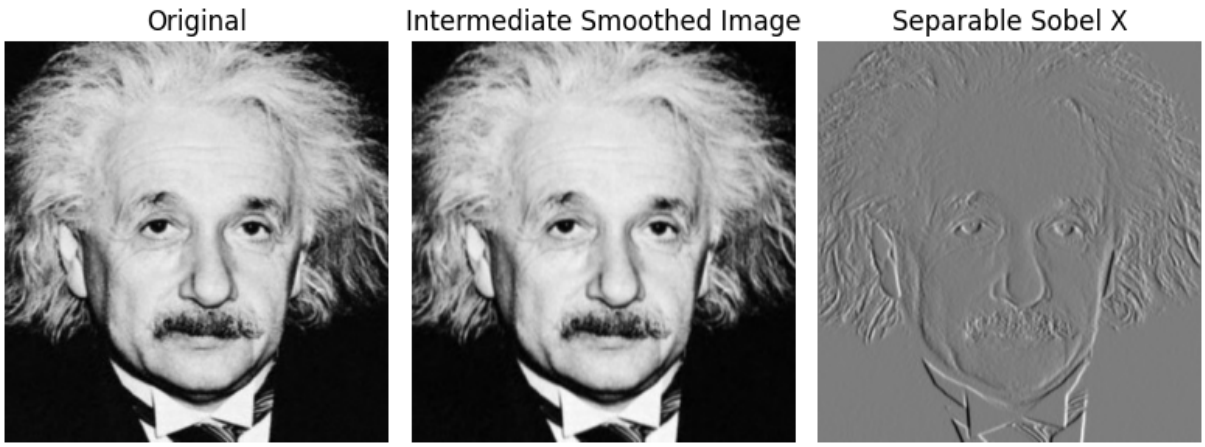
smooth_col=np.array([[1],[2],[1]],dtype=np.float32); derivative_row=np.array([[1,0,-1]],dtype=np.float32)
temp_x=cv2.filter2D(img7,cv2.CV_64F,smooth_col); sobel_x_separable=cv2.filter2D(temp_x,cv2.CV_64F,derivative_row)
fig,axes=plt.subplots(1,3,figsize=(8,4)); axes[0].imshow(img7,cmap='gray'); axes[0].set_title('Original'); axes[1].
 ↪imshow(temp_x,cmap='gray'); axes[1].set_title('Intermediate Smoothed Image'); axes[2].
 ↪imshow(sobel_x_separable,cmap='gray'); axes[2].set_title('Separable Sobel X')
for ax in axes: ax.axis('off')
plt.tight_layout(); plt.show()

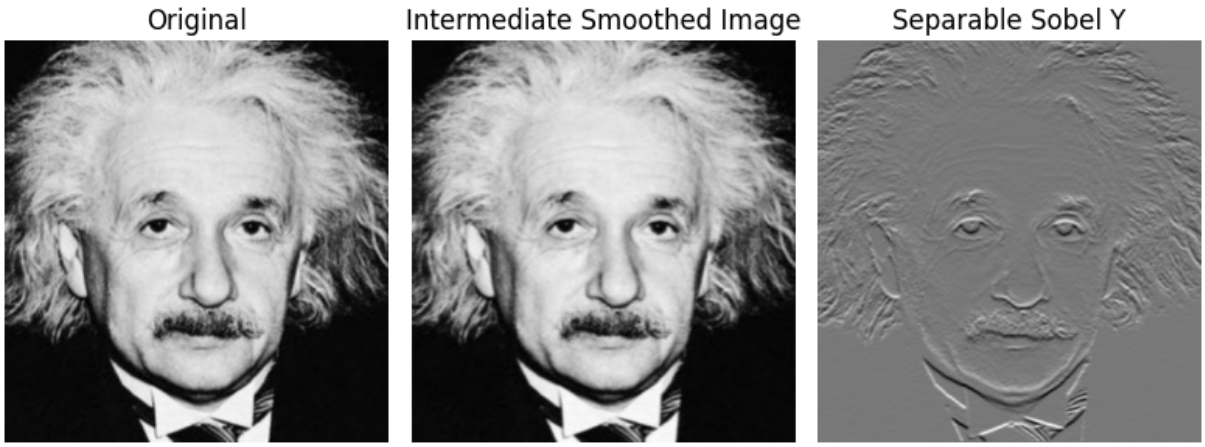
smooth_row=np.array([[1,2,1]],dtype=np.float32); derivative_col=np.array([[1],[0],[-1]],dtype=np.float32)
temp_y=cv2.filter2D(img7,cv2.CV_64F,smooth_row); sobel_y_separable=cv2.filter2D(temp_y,cv2.CV_64F,derivative_col)
```

```
fig,axes=plt.subplots(1,3,figsize=(8,4)); axes[0].imshow(img7,cmap='gray'); axes[0].set_title('Original'); axes[1].
 ↪imshow(temp_y,cmap='gray'); axes[1].set_title('Intermediate Smoothed Image'); axes[2].
 ↪imshow(sobel_y_separable,cmap='gray'); axes[2].set_title('Separable Sobel Y')
for ax in axes: ax.axis('off')
plt.tight_layout(); plt.show()
```









1.8 Question 08

Function to execute zooming using nearest neighbour and bilinear transformation

[21]:
```
def zoom(img, factor, interpolation): #using nearest or bilinear interpolation
    if interpolation == 'nearest':
        zoomed = cv2.resize(img, None, fx=factor, fy=factor, interpolation=cv2.INTER_NEAREST)
    elif interpolation == 'bilinear':
        zoomed = cv2.resize(img, None, fx=factor, fy=factor, interpolation=cv2.INTER_LINEAR)
    else:
        raise ValueError("Interpolation method must be 'nearest' or 'bilinear'.")
    return zoomed

def Mean_Squared_Error(img1,img2):
    h=min(img1.shape[0],img2.shape[0]); w=min(img1.shape[1],img2.shape[1])
    img1_crop=img1[:h,:w]; img2_crop=img2[:h,:w]
    mse=np.mean((img1_crop.astype(np.float64)-img2_crop.astype(np.float64))**2)
    return mse

def show_zoom_comparison(small, original, number):
    nearest = zoom(small, 4, 'nearest')
    bilinear = zoom(small, 4, 'bilinear')
    mse_nearest = Mean_Squared_Error(nearest, original)
    mse_bilinear = Mean_Squared_Error(bilinear, original)
    fig, axes = plt.subplots(1, 3, figsize=(9, 2.5))
    axes[0].imshow(original, cmap='gray'); axes[0].set_title(f'Original Image {number:02d}')
    axes[1].imshow(nearest, cmap='gray'); axes[1].set_title(f'Nearest Neighbor Interpolation\nMSE: {mse_nearest:.2f}')
    axes[2].imshow(bilinear, cmap='gray'); axes[2].set_title(f'Bilinear Interpolation\nMSE: {mse_bilinear:.2f}')
    for ax in axes: ax.axis('off')
    plt.tight_layout(); plt.show()

# Image 01
img01small = load_image("im01small.png", cv2.IMREAD_COLOR_RGB)

img01 = load_image("im01.png", cv2.IMREAD_COLOR_RGB)

# Image 02
img02small = load_image("im02small.png", cv2.IMREAD_COLOR_RGB)

img02 = load_image("im02.png", cv2.IMREAD_COLOR_RGB)

# Image 03
img03small = load_image("im03small.png", cv2.IMREAD_COLOR_RGB)

img03 = load_image("im03.png", cv2.IMREAD_COLOR_RGB)

# Image 04
img04small = load_image("im04small.jpg", cv2.IMREAD_COLOR_RGB)

img04 = load_image("im04.jpg", cv2.IMREAD_COLOR_RGB)

for number, small, original in (
```

```
    (1, img01small, img01),
    (2, img02small, img02),
    (3, img03small, img03),
    (4, img04small, img04),
):
    show_zoom_comparison(small, original, number)
```









1.9 Question 09

[22]:
```
img9 = load_image("q9.jpeg", cv2.IMREAD_COLOR_BGR)

mask = np.zeros(img9.shape[:2], np.uint8)

bg_model = np.zeros((1, 65), np.float64)
fg_model = np.zeros((1, 65), np.float64)

rect = (50, 150, 500, 400)

cv2.grabCut(img9, mask, rect, bg_model, fg_model, 5, cv2.GC_INIT_WITH_RECT)

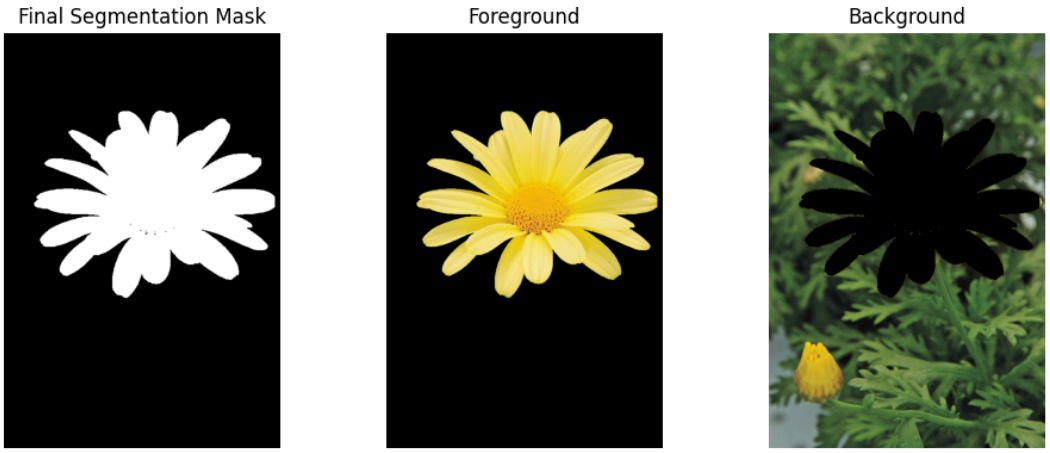
binary_mask = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 255, 0).astype("uint8")
foreground = cv2.bitwise_and(img9, img9, mask=binary_mask)

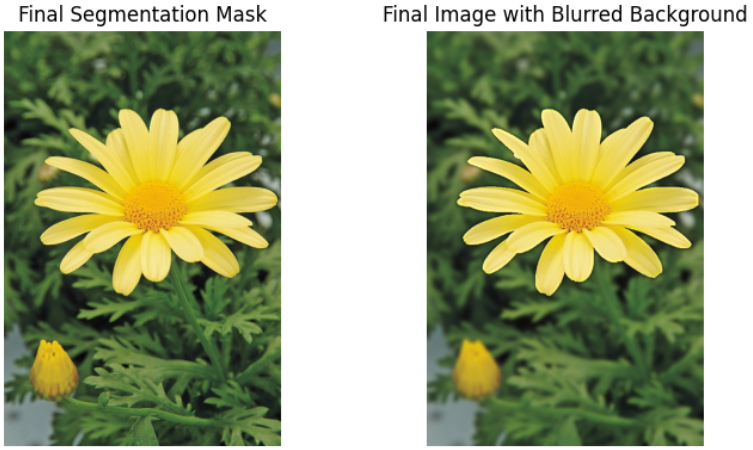
background_mask = cv2.bitwise_not(binary_mask)
background = cv2.bitwise_and(img9, img9, mask=background_mask)

fig,axes=plt.subplots(1,3,figsize=(10,4)); axes[0].imshow(binary_mask,cmap="gray"); axes[0].set_title("Final␣
 ↪Segmentation Mask"); axes[1].imshow(cv2.cvtColor(foreground,cv2.COLOR_BGR2RGB)); axes[1].set_title("Foreground");␣
 ↪axes[2].imshow(cv2.cvtColor(background,cv2.COLOR_BGR2RGB)); axes[2].set_title("Background")
for ax in axes: ax.axis("off")
plt.tight_layout(); plt.show()

background_blurred = cv2.GaussianBlur(background, (21, 21), 0)
final_image = cv2.add(foreground, background_blurred)

fig,axes=plt.subplots(1,2,figsize=(8,4)); axes[0].imshow(cv2.cvtColor(img9,cv2.COLOR_BGR2RGB)); axes[0].
 ↪set_title("Final Segmentation Mask"); axes[1].imshow(cv2.cvtColor(final_image,cv2.COLOR_BGR2RGB)); axes[1].
 ↪set_title("Final Image with Blurred Background")
for ax in axes: ax.axis("off")
plt.tight_layout(); plt.show()
```





The dark outline like edge in the enhanced image is due to the gaussian blur that is used to blur the background image. After using bitwise_and function and getting the segmentation, the foreground part is in black. And when gaussian blurred, this black flower like area also gets blurred, thus, the black colour bleeds into the nearby pixels, creating the black outline-like edge.

1.10 Question 10

[23]:
```
img10 = load_image("q10.jpeg", cv2.IMREAD_COLOR_BGR)

bilateral = cv2.bilateralFilter(img10, d=9, sigmaColor=35, sigmaSpace=30)
gaussian = cv2.GaussianBlur(img10, (3, 3), 2)

fig,axes=plt.subplots(1,3,figsize=(10,4)); axes[0].imshow(cv2.cvtColor(img10,cv2.COLOR_BGR2RGB)); axes[0].
 ↪set_title("Original Image"); axes[1].imshow(cv2.cvtColor(bilateral,cv2.COLOR_BGR2RGB)); axes[1].set_title("Bilateral␣
 ↪Filtered Image"); axes[2].imshow(cv2.cvtColor(gaussian,cv2.COLOR_BGR2RGB)); axes[2].set_title("Gaussian Filtered␣
 ↪Image")
for ax in axes: ax.axis("off")
plt.tight_layout(); plt.show()

import numpy as np

def bilateral_filter_color(img, kernel_size, sigma_s, sigma_r):

    img = img.astype(np.float64)
    h, w, c = img.shape
    pad = kernel_size // 2

    padded = np.pad(img, ((pad,pad),(pad,pad),(0,0)), mode='reflect')
    output = np.zeros_like(img)

    x, y = np.mgrid[-pad:pad+1, -pad:pad+1]
    spatial = np.exp(-(x**2+y**2)/(2*sigma_s**2))

    for i in range(h):
        for j in range(w):

            region = padded[i:i+kernel_size, j:j+kernel_size]
            diff = region - img[i,j]

            colour_dist = np.sum(diff**2, axis=2)
            range_weight = np.exp(-colour_dist/(2*sigma_r**2))

            weights = spatial * range_weight
            weights /= np.sum(weights)

            output[i,j] = np.sum(region * weights[:,:,None], axis=(0,1))

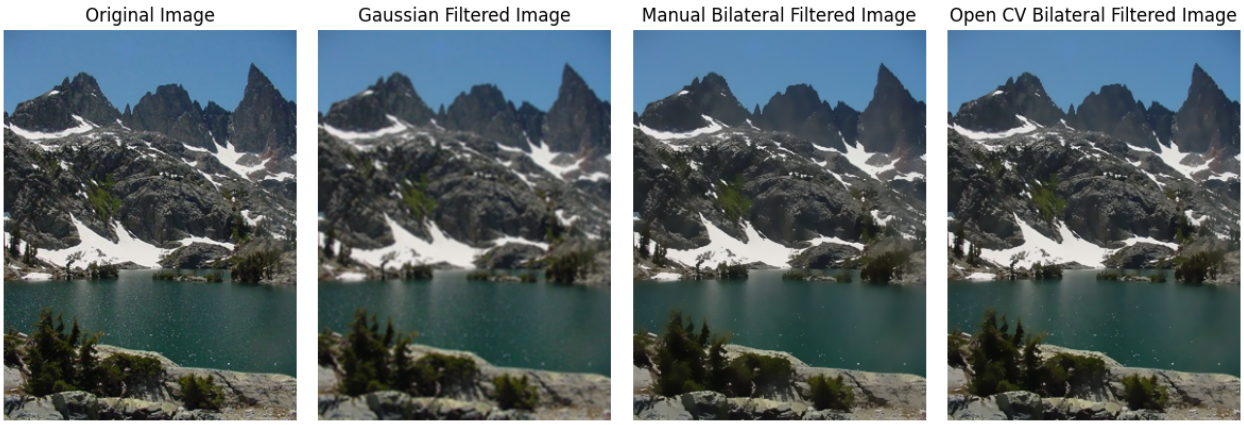
    return np.clip(output,0,255).astype(np.uint8)

manual_bilateral = bilateral_filter_color(img10, kernel_size=9, sigma_s=35, sigma_r=30)
opencv_bilateral = cv2.bilateralFilter(img10, 9, 35, 30)
mse = np.mean((manual_bilateral.astype(float) - opencv_bilateral.astype(float))**2)
print("MSE:", mse)

fig,axes=plt.subplots(1,4,figsize=(12,4)); axes[0].imshow(cv2.cvtColor(img10,cv2.COLOR_BGR2RGB)); axes[0].
 ↪set_title("Original Image"); axes[1].imshow(cv2.cvtColor(gaussian,cv2.COLOR_BGR2RGB)); axes[1].set_title("Gaussian␣
 ↪Filtered Image"); axes[2].imshow(cv2.cvtColor(manual_bilateral,cv2.COLOR_BGR2RGB)); axes[2].set_title("Manual␣
 ↪Bilateral Filtered Image"); axes[3].imshow(cv2.cvtColor(opencv_bilateral,cv2.COLOR_BGR2RGB)); axes[3].
 ↪set_title("Open CV Bilateral Filtered Image")
for ax in axes: ax.axis("off")
plt.tight_layout(); plt.show()
```




```
MSE: 6.743056325023084
```



The difference between the Gaussian and the bilateral filter is that the Gaussian filter considers only the distance between pixels when deciding the contribution of neighbouring pixels. The bilateral filter, on the other hand, considers both the distance between pixels and the intensity difference between them. This allows the bilateral filter to smooth similar regions while preserving important edges, unlike the Gaussian filter which tends to blur edges as well.
MSE : 6.743